

GitHub Enterprise Server Runner to Databricks CI/CD Deployment

Security Architecture Documentation

Document Version: 1.0

Last Updated: February 2026

Purpose: Security review and approval for automated deployment of Databricks Tasks & Jobs via GHES runners

Table of Contents

1. [Executive Summary](#)
 2. [MLOps Overview and Context](#)
 3. [Architecture Overview](#)
 4. [Authentication & Authorization](#)
 5. [Network Security](#)
 6. [Secret Management](#)
 7. [Data Security](#)
 8. [Access Control & Governance](#)
 9. [Audit, Logging & Monitoring](#)
 10. [Supply Chain Security](#)
 11. [Compliance & Best Practices](#)
 12. [Incident Response & Recovery](#)
 13. [Security Checklist](#)
-

1. Executive Summary

Purpose

This document outlines the security architecture for deploying Databricks Tasks and Jobs through GitHub Enterprise Server (GHES) runners hosted on Azure Kubernetes Service (AKS). The implementation enables MLOps workflows while maintaining enterprise security standards.

Scope

- **In Scope:** Authentication mechanisms, network security, secret management, MLOps deployment workflows, access controls, and monitoring for Databricks job deployments
- **Out of Scope:** GHES runner infrastructure setup on AKS (already implemented)

Key Security Principles

- **Zero Trust Architecture:** No implicit trust; verify every access request
 - **Least Privilege Access:** Minimal permissions necessary for each component
 - **Defense in Depth:** Multiple layers of security controls
 - **Audit Everything:** Comprehensive logging and monitoring
 - **Secrets Rotation:** Regular automated rotation of credentials
-

2. MLOps Overview and Context

What is MLOps?

MLOps (Machine Learning Operations) is the practice of applying DevOps principles to machine learning workflows. It encompasses:

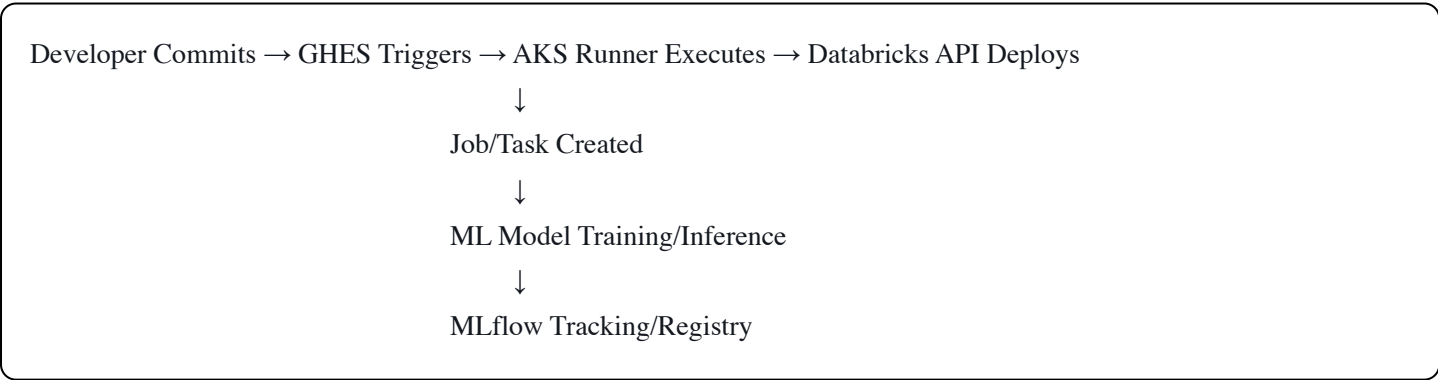
- **Automated ML Pipelines:** Version-controlled, reproducible training and deployment
- **Model Lifecycle Management:** Tracking models from development through production
- **Continuous Training:** Automated retraining with new data
- **Model Monitoring:** Performance tracking and drift detection
- **Governance:** Model versioning, lineage, and approval workflows

MLOps in This Implementation

Our CI/CD pipeline enables:

1. **Automated Deployment:** Databricks jobs and tasks deployed via Git commits
2. **Environment Consistency:** Infrastructure-as-Code ensures reproducible environments
3. **Model Versioning:** Integration with MLflow for model registry and tracking
4. **Validation Gates:** Automated testing before production deployment
5. **Rollback Capabilities:** Version control enables quick rollback to previous states

MLOps Workflow

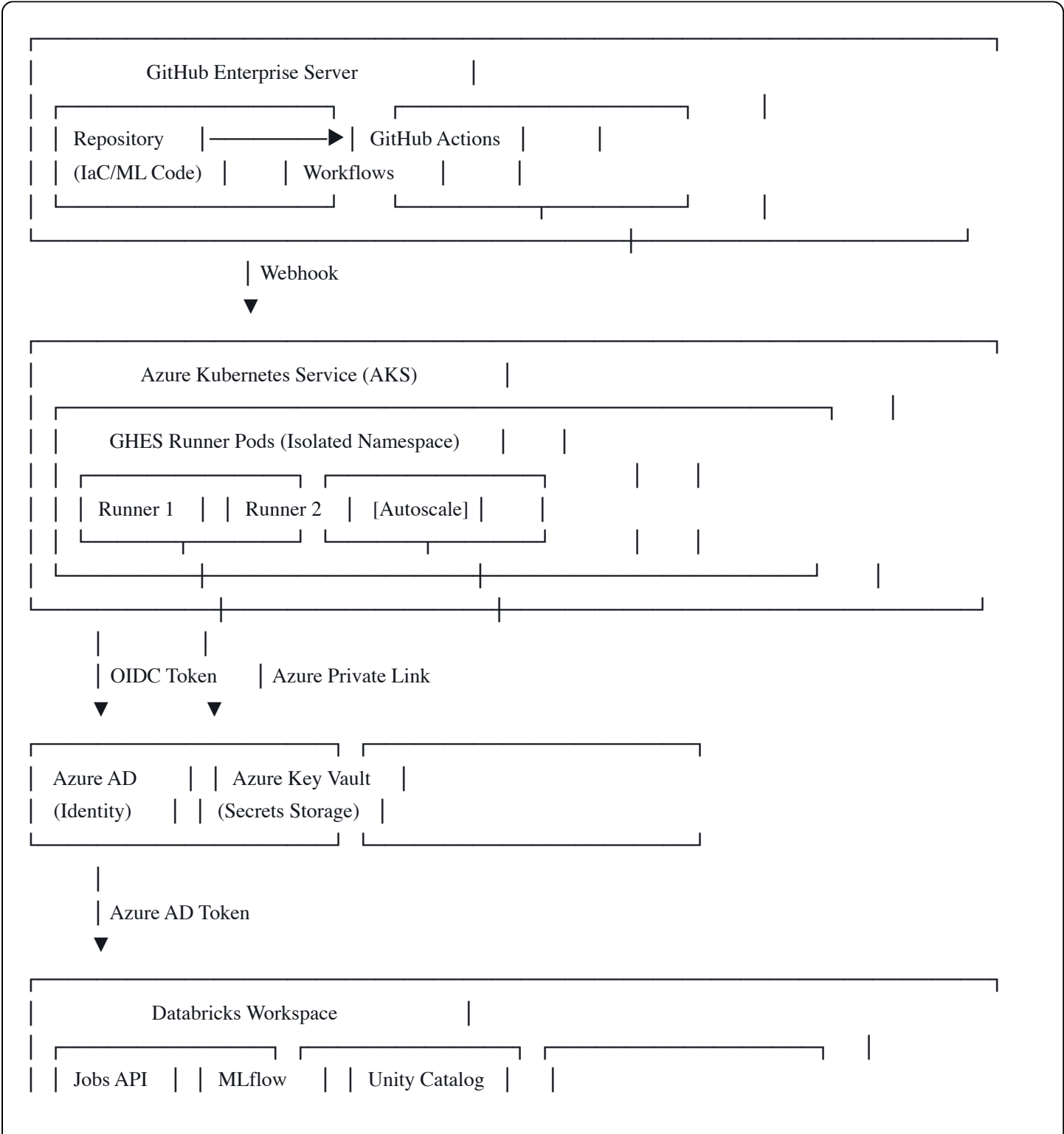


Security Benefits of MLOps

- **Reproducibility:** Eliminates manual deployment errors
- **Auditability:** Complete history of model deployments
- **Compliance:** Automated validation and approval workflows
- **Isolation:** Separate environments (dev/staging/prod) with appropriate controls

3. Architecture Overview

High-Level Component Diagram



Data Flow

1. **Trigger:** Code push/PR triggers GitHub Actions workflow
 2. **Runner Assignment:** GHES assigns job to available AKS runner pod
 3. **Authentication:** Runner authenticates via OIDC to Azure AD
 4. **Secret Retrieval:** Azure AD token used to access Key Vault secrets
 5. **Databricks API Call:** Runner uses Databricks token to deploy job/task
 6. **Execution:** Databricks job runs in workspace cluster
 7. **Logging:** All actions logged to Azure Monitor and Databricks audit logs
-

4. Authentication & Authorization

4.1 OpenID Connect (OIDC) Authentication

Recommendation: Use OIDC federation between GHES and Azure AD (no long-lived secrets)

Configuration

```
yaml

# GitHub Actions Workflow
permissions:
  id-token: write # Required for OIDC token
  contents: read

jobs:
  deploy:
    runs-on: self-hosted
    steps:
      - name: Azure Login via OIDC
        uses: azure/login@v1
        with:
          client-id: ${ secrets.AZURE_CLIENT_ID }
          tenant-id: ${ secrets.AZURE_TENANT_ID }
          subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }
```

Security Benefits

- **No Static Secrets:** Short-lived tokens (1 hour default)
- **Scoped Access:** Tokens bound to specific workflow, repository, and branch

- **Automatic Rotation:** New token for each workflow run
- **Reduced Attack Surface:** No credentials stored in GitHub secrets

4.2 Azure AD Service Principal Configuration

Federated Identity Credential Setup:

```
json

{
  "name": "ghes-databricks-cicd",
  "issuer": "https://token.actions.githubusercontent.com",
  "subject": "repo:organization/repository:environment:production",
  "audiences": ["api://AzureADTokenExchange"]
}
```

Required Claims:

- `repository`: Restrict to specific repo
- `ref`: Restrict to specific branch (e.g., `refs/heads/main`)
- `environment`: Restrict to specific GitHub environment

4.3 Databricks Authentication Options

Option A: Azure AD Passthrough (Recommended)

```
bash

# Service Principal authenticates to Databricks via Azure AD
az account get-access-token \
  --resource 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d \
  --query accessToken
```

Advantages:

- Unified identity management
- Conditional Access policies apply
- MFA enforcement possible
- Centralized audit logs

Option B: Databricks Personal Access Token (PAT)

Security Controls:

- Store in Azure Key Vault, never in GitHub secrets
- Set expiration (max 90 days recommended)

- Scope to minimum permissions (e.g., `jobs:write`, `repos:read`)
- Use separate tokens for dev/staging/prod

Option C: Service Principal Direct Registration

Register Azure AD Service Principal in Databricks workspace:

```
bash

databricks service-principals create \
  --application-id <azure-ad-app-id> \
  --display-name "GHES-CICD-ServicePrincipal"
```

Assign minimal permissions:

- `CAN_MANAGE` on specific job folders
- `CAN_USE` on required clusters/warehouses
- No workspace admin rights

4.4 Authentication Flow

```
1. GHES Runner requests OIDC token from GitHub
↓
2. GitHub issues JWT with repository/branch claims
↓
3. Runner presents JWT to Azure AD
↓
4. Azure AD validates JWT, issues Azure access token
↓
5. Runner uses token to access Key Vault
↓
6. Key Vault returns Databricks credentials (if using PAT)
   OR
   Runner uses Azure AD token directly for Databricks (if using AAD passthrough)
↓
7. Runner calls Databricks Jobs API
↓
8. Databricks validates token/credentials
↓
9. Job/Task deployed
```

5. Network Security

5.1 Network Segmentation

AKS Cluster Network Isolation

- **Dedicated Subnet:** GHES runners in isolated subnet with NSG rules
- **Network Policies:** Kubernetes NetworkPolicy enforcing pod-to-pod restrictions
- **No Public IPs:** Runner pods use private IPs only

Example Network Policy:

```
yaml

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ghes-runner-network-policy
  namespace: github-runners
spec:
  podSelector:
    matchLabels:
      app: ghes-runner
  policyTypes:
    - Egress
  egress:
    # Allow DNS
    - to:
      - namespaceSelector: {}
      ports:
        - protocol: UDP
          port: 53
    # Allow HTTPS to Azure services
    - to:
      - namespaceSelector: {}
      ports:
        - protocol: TCP
          port: 443
    # Allow Databricks API (private endpoint)
    - to:
      - podSelector: {}
      ports:
        - protocol: TCP
          port: 443
```

5.2 Private Endpoints

Azure Key Vault Private Endpoint

- **Configuration:** Private endpoint in AKS VNet

- **DNS Resolution:** Azure Private DNS zone for `vault.azure.net`
- **Firewall:** Key Vault firewall allows only AKS subnet
- **No Public Access:** Public network access disabled

Databricks Private Link (if applicable)

Options:

1. **Backend Private Link:** Secure control plane traffic
2. **Frontend Private Link:** Secure workspace UI access (optional for CI/CD)

Configuration:

- Private endpoint in AKS VNet
- NSG rules allowing HTTPS (443) from runner subnet
- Databricks secure cluster connectivity enabled

5.3 Egress Control

Azure Firewall / NAT Gateway

Allowed Outbound Destinations:

- `login.microsoftonline.com` - Azure AD authentication
- `*.vault.azure.net` - Azure Key Vault
- `*.azuredatabricks.net` - Databricks API (or private endpoint)
- `github.com` - GitHub API (for workflow downloads)
- `*.blob.core.windows.net` - Azure Blob Storage (if needed)

Blocked by Default:

- All other internet destinations
- Implement explicit allowlist

DNS Security

- **Private DNS Zones:** For Azure services
- **DNS Filtering:** Block known malicious domains
- **DNS Logging:** Log all DNS queries for threat detection

5.4 TLS/SSL Configuration

Requirements:

- **TLS 1.2 Minimum:** Disable older protocols

- **Certificate Validation:** Verify certificates for all HTTPS connections
- **Certificate Pinning:** (Optional) Pin Databricks API certificate

Example:

```
python

# Databricks SDK configuration
from databricks.sdk import WorkspaceClient

w = WorkspaceClient(
    host="https://<workspace>.azuredatabricks.net",
    azure_client_id=os.getenv("AZURE_CLIENT_ID"),
    azure_client_secret=os.getenv("AZURE_CLIENT_SECRET"),
    azure_tenant_id=os.getenv("AZURE_TENANT_ID"),
    # Enforce TLS 1.2+
    http_timeout_seconds=60,
    retry_timeout_seconds=300
)
```

5.5 Network Monitoring

- **Azure Network Watcher:** Monitor network flows from AKS
- **NSG Flow Logs:** Capture all NSG allow/deny decisions
- **Traffic Analytics:** Identify anomalous traffic patterns
- **DDoS Protection:** Azure DDoS Protection Standard (if applicable)

6. Secret Management

6.1 Azure Key Vault Integration

Secret Storage Architecture

```
Azure Key Vault (Premium SKU)
├── Secrets
│   ├── databricks-host
│   ├── databricks-token (if not using Azure AD)
│   ├── mlflow-tracking-uri
│   └── additional-api-keys
├── Access Policies / RBAC
│   └── Service Principal: Get Secret permissions only
└── Audit Logs → Log Analytics Workspace
```

Key Vault Configuration

Security Settings:

- **Soft Delete:** Enabled (90-day retention)
- **Purge Protection:** Enabled
- **Public Access:** Disabled
- **Private Endpoint:** Enabled
- **Firewall:** Allow AKS subnet only

Access Control:

```
bash

# RBAC Assignment (Recommended over Access Policies)
az role assignment create \
  --role "Key Vault Secrets User" \
  --assignee <service-principal-id> \
  --scope /subscriptions/<sub-id>/resourceGroups/<rg>/providers/Microsoft.KeyVault/vaults/<kv-name>
```

6.2 Databricks Secret Scopes

Azure Key Vault-Backed Secret Scope:

```
bash

# Create secret scope backed by Azure Key Vault
databricks secrets create-scope \
  --scope azure-key-vault-scope \
  --scope-backend-type AZURE_KEYVAULT \
  --resource-id /subscriptions/<sub-id>/resourceGroups/<rg>/providers/Microsoft.KeyVault/vaults/<kv-name> \
  --dns-name <kv-name>.vault.azure.net
```

Advantages:

- Secrets managed in Key Vault (single source of truth)
- Automatic sync with Databricks
- Key Vault RBAC controls access
- Audit logs in Azure Monitor

Access Control:

- **MANAGE:** Only CI/CD service principal
- **READ:** Databricks clusters/jobs that need secrets
- **WRITE:** Not granted to any runtime entities

6.3 Secret Rotation

Automated Rotation Strategy

1. **Databricks Tokens:** Rotate every 60-90 days
2. **Service Principal Secrets:** Rotate every 90 days (if not using OIDC)
3. **Rotation Process:**
 - Create new secret
 - Update Key Vault
 - Test with new secret
 - Revoke old secret
 - Document in change log

GitHub Actions Rotation Workflow:

```
yaml
name: Rotate Databricks Token
on:
  schedule:
    - cron: '0 0 1 */2 *' # Every 2 months
  workflow_dispatch:

jobs:
  rotate:
    runs-on: self-hosted
    steps:
      - name: Generate New Token
        # Script to create new Databricks token
      - name: Update Key Vault
        # Update secret in Azure Key Vault
      - name: Revoke Old Token
        # Revoke previous token in Databricks
```

6.4 Secret Access Workflow

```
yaml
```

```
# In GitHub Actions
- name: Retrieve Databricks Token
  run: |
    # Login via OIDC (no secrets needed)
    az login --federated-token $ACTIONS_ID_TOKEN_REQUEST_TOKEN \
      --service-principal \
      -u $AZURE_CLIENT_ID \
      -t $AZURE_TENANT_ID

    # Get secret from Key Vault
    DATABRICKS_TOKEN=$(az keyvault secret show \
      --vault-name ${{ secrets.KEY_VAULT_NAME }} \
      --name databricks-token \
      --query value -o tsv)

    # Use in subsequent steps (never echo/log)
    echo "::add-mask::$DATABRICKS_TOKEN"
```

6.5 Secret Security Best Practices

- **Never Log Secrets:** Use GitHub Actions `add-mask` feature
 - **Limit Scope:** Use separate secrets for dev/staging/prod
 - **Expiration:** Set expiration on all secrets
 - **Monitoring:** Alert on secret access patterns
 - **Least Privilege:** Only grant access to specific secrets needed
-

7. Data Security

7.1 Data in Transit

Encryption Standards:

- **GitHub to AKS:** HTTPS with TLS 1.2+
- **AKS to Azure AD:** HTTPS with TLS 1.2+
- **AKS to Key Vault:** HTTPS over private endpoint
- **AKS to Databricks:** HTTPS with TLS 1.2+ (over private link if enabled)
- **Databricks Internal:** TLS 1.2+ for all cluster communications

Certificate Management:

- **Azure-Managed Certificates:** For Azure services
- **Let's Encrypt / CA:** For custom domains

- **Certificate Rotation:** Automated via Azure

7.2 Data at Rest

Azure Key Vault

- **Encryption:** Hardware Security Module (HSM) backed
- **Key Management:** Microsoft-managed or customer-managed keys (CMK)

Databricks Storage

- **DBFS Encryption:** Enabled by default (Azure-managed keys)
- **Unity Catalog:** Supports customer-managed keys
- **MLflow Artifacts:** Stored in encrypted Azure Blob Storage

Recommendation: Use customer-managed keys for production workspaces

7.3 Code and Artifact Security

GitHub Repository

- **Branch Protection:**
 - Require pull request reviews
 - Require status checks (automated tests)
 - No force push / deletion allowed
 - Require signed commits (optional)
- **Code Scanning:**
 - GitHub Advanced Security (dependency scanning, secret scanning)
 - CodeQL analysis for vulnerabilities

Databricks Repos

- **Git Integration:** Link to GitHub Enterprise repository
- **Access Control:** Separate read/write permissions
- **No Secrets in Code:** Use secret scopes, not hardcoded values

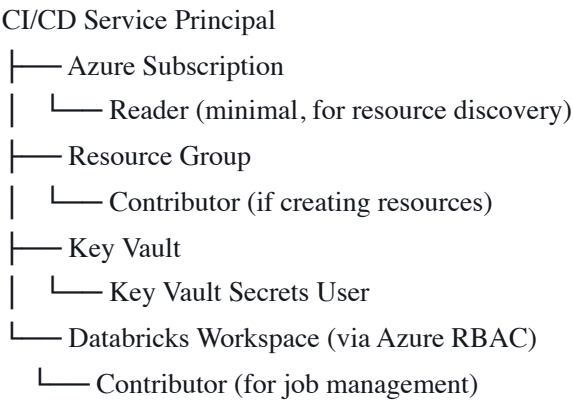
MLflow Model Artifacts

- **Storage:** Azure Blob Storage with lifecycle policies
- **Encryption:** Storage Service Encryption (SSE) enabled
- **Access Control:** Managed Identity / SAS tokens with expiration
- **Versioning:** Blob versioning enabled for rollback

8. Access Control & Governance

8.1 Role-Based Access Control (RBAC)

Azure AD Roles



Databricks Workspace Roles

Service Principal Assignments:

- **Account Admin:** No (never for automation)
- **Workspace Admin:** No (never for automation)
- **Custom Role:**
 - `jobs:create`, `jobs:update`, `jobs:delete`
 - `clusters:create` (if needed for job clusters)
 - `repos:read`, `repos:write`
 - `mlflow:write` (for model registration)

Unity Catalog Permissions:

- `USE CATALOG` on specific catalog
- `USE SCHEMA` on specific schema
- `SELECT` / `MODIFY` on specific tables (as needed)
- No `CREATE CATALOG` or `CREATE SCHEMA`

8.2 Environment Separation

GitHub Environments

yaml

Repository Environments

```
|— development
|   |— Required reviewers: None
|   |— Deployment branches: develop
|   |— Secrets: DEV_* variables
|— staging
|   |— Required reviewers: 1 team member
|   |— Deployment branches: staging
|   |— Secrets: STG_* variables
|— production
|   |— Required reviewers: 2 team members + security
|   |— Deployment branches: main only
|   |— Secrets: PROD_* variables
```

OIDC Subject Claims by Environment:

```
json

{
  "development": "repo:org/repo:environment:development",
  "staging": "repo:org/repo:environment:staging",
  "production": "repo:org/repo:environment:production:ref:refs/heads/main"
}
```

Databricks Workspaces

- **Separate Workspaces:** Dev, Staging, Production (recommended)
- **OR Separate Catalogs/Schemas:** In Unity Catalog
- **Network Isolation:** Separate VNets/Subnets per environment (optional)
- **Service Principals:** Separate per environment

8.3 Just-In-Time (JIT) Access

Implementation:

- **Privileged Identity Management (PIM):** For admin access to Azure resources
- **Time-Bound Permissions:** Temporary elevation for emergency fixes
- **Approval Workflow:** Require justification and approval

8.4 Conditional Access Policies

Azure AD Conditional Access:

- **Device Compliance:** Require managed devices for admin access
- **IP Restrictions:** Limit admin access to corporate network

- **MFA:** Require for interactive admin sessions
- **Risk-Based:** Block high-risk sign-ins

Note: Service principal (OIDC) authentication bypasses some conditional access policies - use federated credential claims for restrictions

9. Audit, Logging & Monitoring

9.1 Comprehensive Logging Strategy

Azure Activity Logs

- **Subscription Level:** All resource changes logged
- **Retention:** Minimum 90 days
- **Destination:** Log Analytics Workspace

Azure Key Vault Diagnostic Logs

Enabled Logs:

- `AuditEvent` - All secret access operations
- `AzurePolicyEvaluationDetails` - Policy compliance
- **Retention:** 365 days minimum
- **Alerting:** Alert on unusual access patterns

Sample Alert Rule:

```
kusto

// Alert on multiple Key Vault secret retrievals
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.KEYVAULT"
| where OperationName == "SecretGet"
| summarize Count=count() by identity_claim_appid_g, bin(TimeGenerated, 5m)
| where Count > 10 // More than 10 secret retrievals in 5 minutes
```

Databricks Audit Logs

Enable Audit Log Delivery:

```
json
```



```
{
  "account_id": "<account-id>",
  "workspace_id": "<workspace-id>",
  "storage_configuration": {
    "type": "AZURE_STORAGE",
    "storage_account_name": "<storage-account>",
    "container_name": "databricks-audit-logs"
  }
}
```

Key Events to Monitor:

- `jobs.create`, `jobs.update`, `jobs.delete`
- `clusters.create`, `clusters.delete`
- `secretScopes.getSecret` (secret access)
- `workspace.download` (code/notebook downloads)
- `authentication.login` (user/SP logins)

GitHub Audit Logs

GHES Audit Log Configuration:

- **Events:** Workflow runs, secret access, repository changes
- **Forwarding:** Send to SIEM (e.g., Azure Sentinel)
- **Retention:** Per GitHub retention policy

9.2 Monitoring & Alerting

Azure Monitor Alerts

Critical Alerts:

1. Unauthorized Access Attempts

- Failed authentication to Key Vault
- Failed Databricks API calls
- Unauthorized IP access attempts

2. Unusual Activity

- Job deployments outside normal hours
- Rapid secret rotation
- Large-scale job modifications

3. Security Events

- New service principal created

- RBAC role assignments changed
- Network security group rule modifications

Example Alert:

```
kusto
// Alert on failed Databricks API calls
AzureDiagnostics
| where Category == "DatabricksWorkspace"
| where httpStatusCode_d >= 400
| where userAgent_s contains "GitHub-Actions"
| summarize FailureCount=count() by bin(TimeGenerated, 15m), identity_claim_appid_g
| where FailureCount > 5
```

Databricks Cluster Monitoring

- **Cluster Events:** Track creation/deletion/termination
- **Job Run Monitoring:** Alert on repeated failures
- **Cost Anomalies:** Detect unexpected compute spend

SIEM Integration

Azure Sentinel Workbooks:

- CI/CD Security Dashboard
- Databricks Threat Detection
- Key Vault Access Patterns
- Anomaly Detection (ML-based)

9.3 Logging Security Best Practices

- **Immutable Logs:** Write to append-only storage
 - **Log Encryption:** Encrypt logs at rest
 - **Access Control:** Restrict log access to security team
 - **Retention:** Comply with regulatory requirements (e.g., 7 years for financial)
 - **PII Redaction:** Mask sensitive data in logs
-

10. Supply Chain Security

10.1 Runner Image Security

Container Image Scanning

Process:

1. Build runner image from official base (e.g., `ubuntu:22.04`)
2. Scan with Trivy/Grype for vulnerabilities
3. Sign image with Cosign/Notary
4. Push to Azure Container Registry
5. AKS pulls only signed images

GitHub Actions Workflow:

```
yaml

- name: Build and Scan Runner Image
  run: |
    docker build -t acr.azurecr.io/ghes-runner:${{ github.sha }} .
    trivy image --severity HIGH,CRITICAL acr.azurecr.io/ghes-runner:${{ github.sha }}

- name: Sign Image
  run: |
    cosign sign --key cosign.key acr.azurecr.io/ghes-runner:${{ github.sha }}
```

AKS Admission Controller:

```
yaml

# Enforce image signature verification
apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: require-signed-images
spec:
  images:
    - glob: "acr.azurecr.io/**"
  authorities:
    - key:
        data: |
          -----BEGIN PUBLIC KEY-----
          <public-key>
          -----END PUBLIC KEY-----
```

10.2 Dependency Management

Python Dependencies (for Databricks)

Requirements.txt Security:

```
txt

# Pin exact versions
databricks-sdk==0.18.0
pyyaml==6.0.1

# Hash verification (pip install --require-hashes)
requests==2.31.0 \
    --hash=sha256:942c5a758f98d7e108b1b21f5f85b2419c8b7c3c9a0b1c6c5d4d4b0e1f5e5e5e
```

Automated Vulnerability Scanning:

- **Dependabot:** Enable on GitHub Enterprise
- **Snyk / WhiteSource:** Continuous monitoring
- **SBOM Generation:** Create Software Bill of Materials

GitHub Actions Pinning

Pin Actions by SHA (not tags):

```
yaml

# Bad - uses tag (mutable)
- uses: actions/checkout@v4

# Good - uses commit SHA (immutable)
- uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11 # v4.1.1
```

10.3 Artifact Integrity

Databricks Job Definitions

Version Control:

- Store job JSON in Git repository
- Tag releases (e.g., `v1.2.3`)
- Generate checksum for artifacts

Integrity Verification:

```
bash
```

```
# Generate SHA256 checksum
sha256sum job-definition.json > job-definition.json.sha256

# Verify before deployment
sha256sum -c job-definition.json.sha256
```

MLflow Model Signing

Sign Models Before Registration:

```
python

import mlflow
from mlflow.models.signature import infer_signature

# Log model with signature
mlflow.sklearn.log_model(
    model,
    "model",
    signature=infer_signature(X_train, y_train),
    registered_model_name="my_model"
)
```

10.4 Third-Party Risk Management

Vendor Assessment:

- **GitHub Enterprise:** SOC 2 Type II certified
- **Azure:** ISO 27001, SOC 2/3, HIPAA, FedRAMP
- **Databricks:** SOC 2 Type II, ISO 27001, HIPAA eligible

Regular Reviews:

- Quarterly review of third-party security posture
- Monitor security bulletins
- Test disaster recovery procedures

11. Compliance & Best Practices

11.1 Regulatory Compliance

GDPR Considerations

- **Data Residency:** Ensure Azure region complies with GDPR

- **Data Subject Rights:** Ability to delete user data
- **Data Processing Agreement:** With Azure and Databricks
- **Audit Logs:** Maintain logs for data access and modifications

SOC 2 / ISO 27001

Control Mappings:

- **Access Control (AC):** RBAC, OIDC, MFA
- **Audit and Accountability (AU):** Comprehensive logging
- **Identification and Authentication (IA):** Azure AD, service principals
- **System and Communications Protection (SC):** Encryption, network security

11.2 Security Frameworks

CIS Benchmarks

Azure Kubernetes Service:

- CIS AKS Benchmark compliance
- Pod Security Standards (restricted profile)

Databricks:

- Enable workspace firewall
- Unity Catalog enabled
- Credential passthrough enabled

NIST Cybersecurity Framework

Map controls to:

- **Identify:** Asset inventory, risk assessment
- **Protect:** Access control, data security
- **Detect:** Logging, monitoring, anomaly detection
- **Respond:** Incident response plan
- **Recover:** Backup and restore procedures

11.3 Secure Development Lifecycle

Pre-Deployment Checks

Automated Security Scanning:

```
yaml
```

```
- name: Security Checks
run: |
  # Secret scanning
  gitleaks detect --source . --report-path gitleaks-report.json

  # Code quality
  pylint databricks_jobs/*.py

  # Terraform security
  tfsec .

  # Dependency vulnerabilities
  safety check -r requirements.txt
```

Manual Review:

- Peer review of infrastructure changes
- Security team approval for production deployments

Post-Deployment Validation

```
yaml
- name: Validate Deployment
run: |
  # Verify job created successfully
  databricks jobs get --job-id <job-id>

  # Check job permissions
  databricks permissions get jobs/<job-id>

  # Test job execution (dry-run)
  databricks jobs run-now --job-id <job-id> --dry-run
```

11.4 Documentation & Training

Required Documentation:

- Runbooks for common scenarios
- Incident response procedures
- Architecture diagrams (this document)
- Change management process

Training:

- Security awareness for developers
 - Secure coding practices
 - Incident response drills (quarterly)
-

12. Incident Response & Recovery

12.1 Incident Response Plan

Detection

Indicators of Compromise:

- Unusual API call patterns
- Failed authentication attempts spike
- Unexpected job/task modifications
- Secret access outside normal hours
- Databricks clusters created in unusual regions

Response Procedures

Severity Levels:

Severity	Description	Response Time	Examples
Critical	Active exploit, data breach	15 minutes	Compromised credentials, unauthorized data access
High	Security control failure	1 hour	Failed authentication spike, unusual job deployments
Medium	Potential security issue	4 hours	Misconfigured permissions, non-compliant resources
Low	Best practice violation	24 hours	Outdated dependencies, missing tags

Response Steps:

1. **Isolate:**
 - Revoke compromised credentials immediately
 - Disable affected service principal
 - Stop running jobs/clusters if necessary
2. **Investigate:**
 - Review audit logs
 - Identify scope of compromise

- Document timeline

3. **Remediate:**

- Rotate all secrets
- Patch vulnerabilities
- Update firewall rules

4. **Recover:**

- Restore from backup if needed
- Re-deploy jobs/tasks
- Validate security controls

5. **Post-Incident:**

- Root cause analysis
- Update procedures
- Security awareness communication

12.2 Backup & Recovery

GitHub Repository Backup

- **Primary:** GitHub Enterprise built-in backups
- **Secondary:** Mirror to Azure Repos (automated sync)
- **Frequency:** Real-time (Git push triggers sync)

Databricks Workspace Backup

Assets to Backup:

- Job definitions (JSON export)
- Cluster configurations
- Notebooks (via Git integration)
- MLflow models (versioned in registry)
- Unity Catalog metadata (if applicable)

Automation:

```
yaml
```

```
# Scheduled backup workflow
name: Backup Databricks Assets
on:
  schedule:
    - cron: '0 2 * * *' # Daily at 2 AM

jobs:
  backup:
    runs-on: self-hosted
    steps:
      - name: Export Jobs
        run: |
          databricks jobs list --output json > jobs-backup.json

      - name: Upload to Azure Blob
        run: |
          az storage blob upload \
            --account-name backupstorage \
            --container-name databricks-backups \
            --name jobs-backup-$(date +%Y%m%d).json \
            --file jobs-backup.json
```

Retention:

- Daily backups: 30 days
- Weekly backups: 90 days
- Monthly backups: 1 year

12.3 Disaster Recovery

Recovery Time Objective (RTO): 4 hours

Recovery Point Objective (RPO): 24 hours

Recovery Procedures:

1. Azure Resource Failure:

- Failover to secondary Azure region (if configured)
- Re-deploy infrastructure via Terraform

2. Databricks Workspace Failure:

- Restore from backup
- Re-register models in MLflow
- Re-deploy jobs from Git repository

3. AKS Cluster Failure:

- AKS auto-healing restarts nodes
- Manual intervention if cluster-wide failure
- Runners auto-register with GHES

Testing:

- Quarterly disaster recovery drills
 - Document lessons learned
 - Update procedures based on findings
-

13. Security Checklist

Pre-Deployment Checklist

Authentication & Authorization

- ☐ OIDC federated identity credentials configured
- ☐ Service principal has minimal required permissions
- ☐ Databricks workspace access control configured
- ☐ Unity Catalog permissions set (if applicable)
- ☐ Conditional access policies reviewed

Network Security

- ☐ AKS subnet NSG rules configured
- ☐ Network policies applied to runner pods
- ☐ Private endpoints created for Key Vault
- ☐ Databricks private link enabled (if required)
- ☐ Egress firewall rules configured

Secret Management

- ☐ All secrets stored in Azure Key Vault
- ☐ Key Vault soft delete and purge protection enabled
- ☐ Key Vault private endpoint configured
- ☐ Databricks secret scope created and linked to Key Vault
- ☐ Secret rotation schedule defined

Data Security

- ☐ TLS 1.2+ enforced for all connections
- ☐ Data at rest encryption enabled
- ☐ Customer-managed keys configured (if required)
- ☐ Storage accounts network restricted

Logging & Monitoring

- ☐ Azure Activity Logs enabled
- ☐ Key Vault diagnostic logs sent to Log Analytics
- ☐ Databricks audit logs configured
- ☐ Alerts configured for critical events
- ☐ SIEM integration tested

Supply Chain Security

- ☐ Runner images scanned for vulnerabilities
- ☐ Dependencies pinned to specific versions
- ☐ GitHub Actions pinned by SHA
- ☐ SBOM generated for dependencies

Compliance

- ☐ Data residency requirements met
- ☐ Regulatory compliance documented
- ☐ Security framework controls mapped
- ☐ Required documentation completed

Incident Response

- ☐ Incident response plan documented
- ☐ Backup procedures tested
- ☐ Disaster recovery plan validated
- ☐ Contact information updated

Post-Deployment Checklist

- ☐ Deployment validated successfully
- ☐ Audit logs reviewed for anomalies
- ☐ Security scanning passed
- ☐ Performance metrics within acceptable range
- ☐ Documentation updated with any changes
- ☐ Stakeholders notified of deployment

Ongoing Maintenance

Monthly:

- ☐ Review access logs for anomalies
- ☐ Update dependencies with security patches
- ☐ Review and update firewall rules
- ☐ Validate backup integrity

Quarterly:

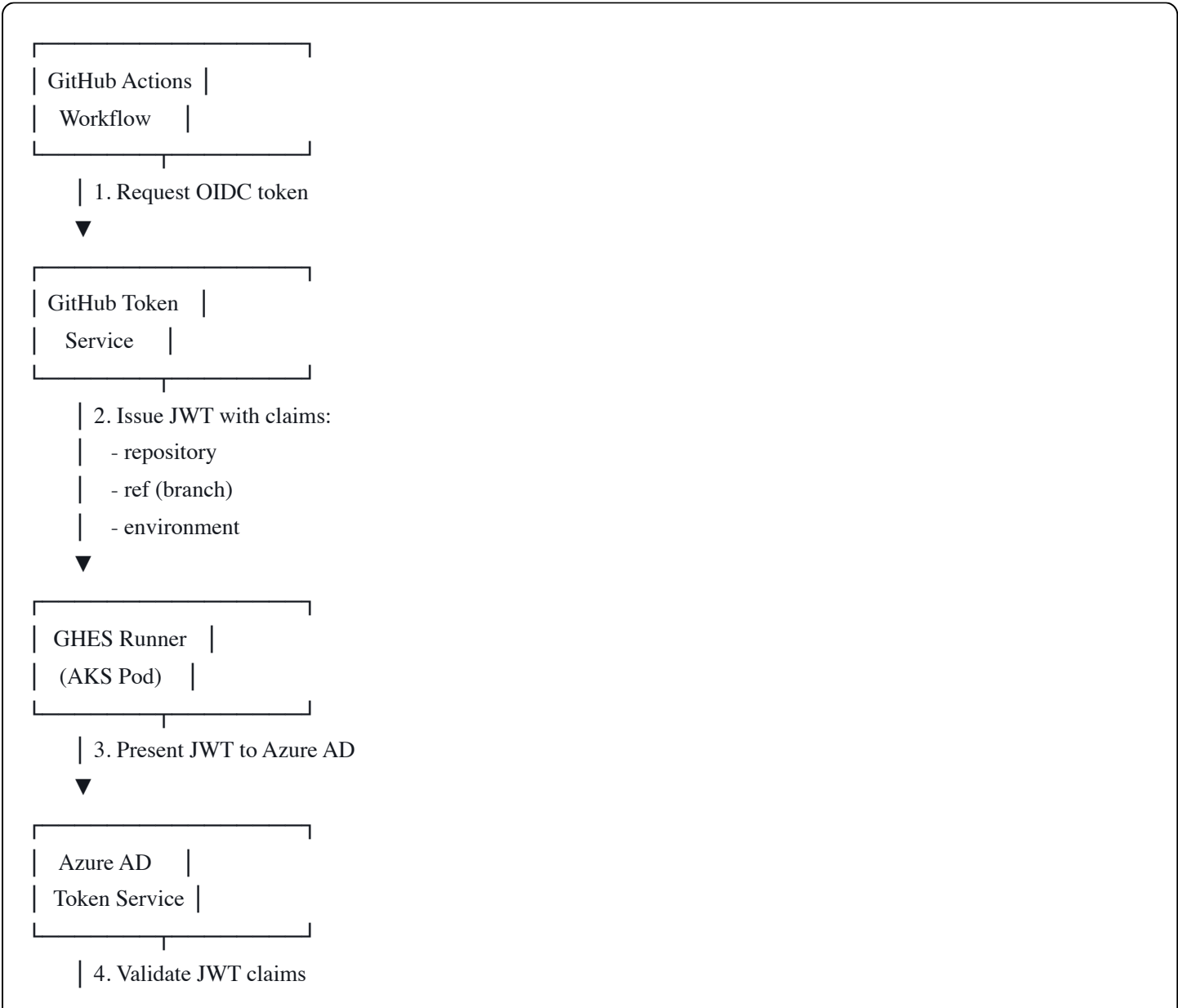
- ☐ Rotate service principal credentials (if not using OIDC)
- ☐ Rotate Databricks tokens
- ☐ Conduct disaster recovery drill
- ☐ Review and update security policies
- ☐ Conduct vulnerability assessment

Annually:

- ☐ Comprehensive security audit
- ☐ Penetration testing
- ☐ Update incident response procedures
- ☐ Review third-party vendor compliance

Appendix A: Reference Architecture Diagrams

Detailed Authentication Flow



5. Issue Azure AD access token (1hr TTL)



GHES Runner

6. Use access token



Azure Key Vault

OR

Databricks API

Network Flow Diagram

Internet



Webhook (HTTPS)



GitHub Enterprise Server



(Private connection / VPN)



Azure Virtual Network

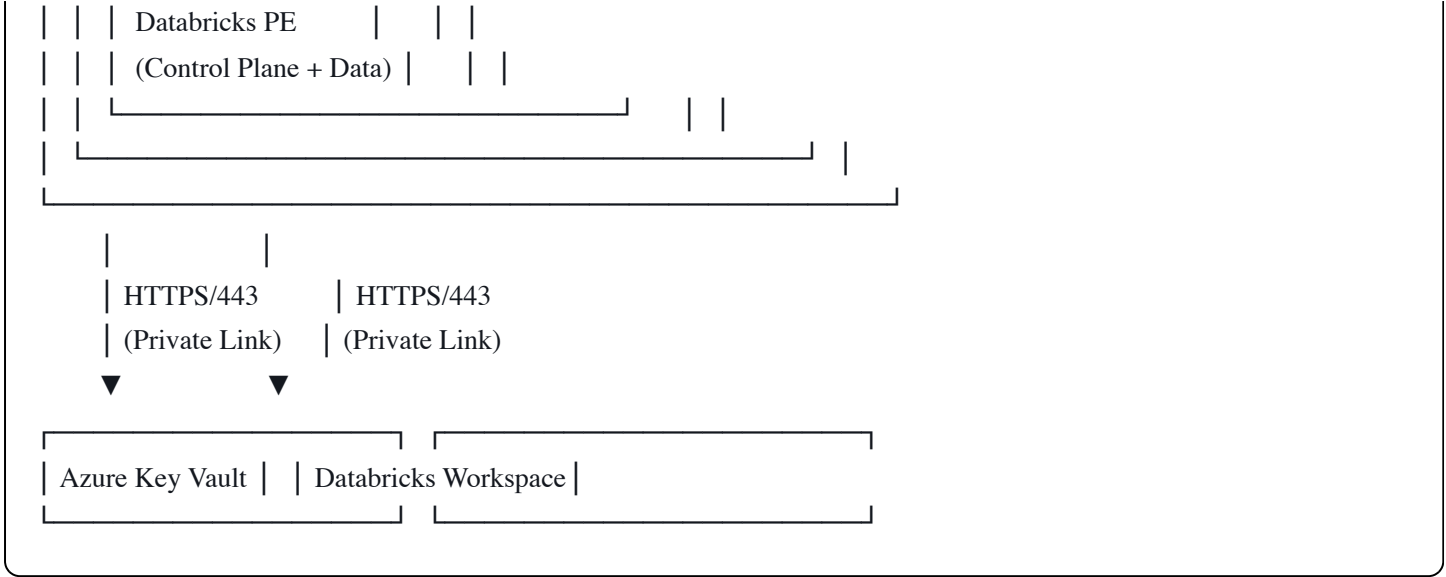
Subnet: AKS Nodes

GHES Runner Pods

(Namespace: github-runners)

Subnet: Private Endpoints

Key Vault PE



Appendix B: Recommended Tools & Resources

Security Scanning Tools

- **Container Scanning:** Trivy, Gype, Clair
- **Secret Scanning:** GitLeaks, TruffleHog, GitHub Advanced Security
- **Code Analysis:** SonarQube, CodeQL, Semgrep
- **Dependency Scanning:** Dependabot, Snyk, WhiteSource
- **Infrastructure Scanning:** tfsec, Checkov, Terraform Sentinel

Monitoring & SIEM

- **Azure Native:** Azure Monitor, Log Analytics, Azure Sentinel
- **Third-Party:** Splunk, Datadog, Elastic Security

Documentation

- **Databricks Security:** <https://docs.databricks.com/security/index.html>
- **Azure Security Baseline:** <https://docs.microsoft.com/en-us/security/benchmark/azure/>
- **GitHub Actions Security:** https://docs.github.com/en/actions/security_guides
- **OIDC in Azure:** <https://docs.microsoft.com/en-us/azure/active-directory/workload-identities/>

Document Approval

Role	Name	Signature	Date
Security Architect			

Role	Name	Signature	Date
Cloud Infrastructure Lead			
Data Platform Lead			
CISO / Security Director			

Document Control

- **Next Review Date:** [6 months from approval]
- **Change History:** Version 1.0 - Initial document creation
- **Feedback:** Submit feedback to [security-team@organization.com]

This document contains confidential information and should be treated as restricted.